Project 3 - Task 1
Text Preprocessing and Feature Extraction

Submitted by: Manish Pawar

Domain: Data Science and ML in Python

Topic: Natural Language Processing and Sentiment Analysis

Objective

The objective of Task 1 is to clean raw movie review text, tokenize it, remove stopwords, apply lemmatization, convert the cleaned text into TF-IDF numerical features, and analyze common words in positive and negative reviews.

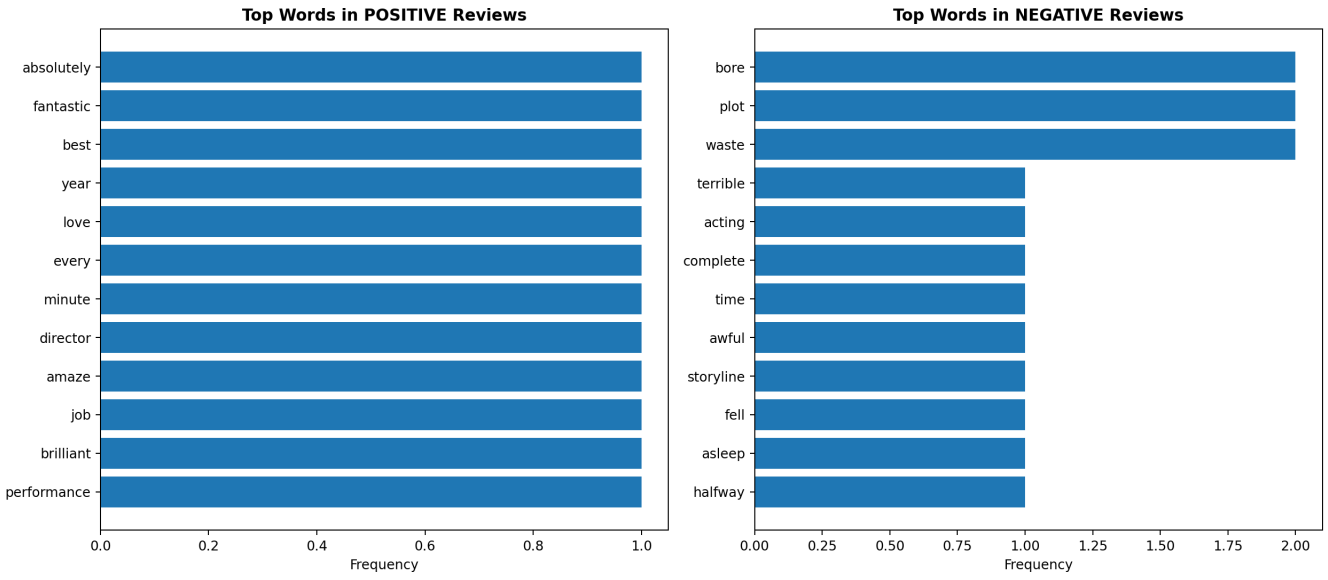
Complete Python Code

```
# Project 3 - Task 1: Text Preprocessing and Feature Extraction
# Name: Manish Pawar

import re
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter
from sklearn.feature_extraction.text import TfidfVectorizer

reviews = [
    ('This movie was absolutely fantastic! Best film of the year.', 'positive'),
    ('Terrible acting and boring plot. Complete waste of time!', 'negative'),
    ('Loved every minute of it. The director did an amazing job.', 'positive'),
    ('Awful storyline. I fell asleep halfway through.', 'negative'),
    ('Brilliant performances! The cinematography was breathtaking.', 'positive'),
    ('Worst movie I have ever seen. Do not waste your money.', 'negative'),
    ('Heartwarming and beautifully crafted. A true masterpiece.', 'positive'),
    ('Dull and predictable. The script was painfully bad.', 'negative'),
]

df = pd.DataFrame(reviews, columns=['review', 'sentiment'])

STOP_WORDS = set("a an and are as at be been being by for from has have he her hers him his i in into is it
LEMMA_MAP = {'performances':'performance','scenes':'scene','characters':'character','visuals':'visual','act

def clean_text(text):
    text = re.sub(r'<[^>]+>', '', text)           # remove HTML tags
    text = re.sub(r'http\S+|www\S+', '', text)   # remove URLs
    text = re.sub(r'[^a-zA-Z\s]', '', text)      # remove punctuation and numbers
    return re.sub(r'\s+', ' ', text).lower().strip()

def tokenize_text(text):
    return re.findall(r'[a-zA-Z]+', text.lower())

def lemmatize(word):
    return LEMMA_MAP.get(word, word)

def preprocess_text(text):
    text = clean_text(text)
    tokens = tokenize_text(text)
    tokens = [t for t in tokens if t not in STOP_WORDS and len(t) > 2]
    tokens = [lemmatize(t) for t in tokens]
    return ' '.join(tokens)

df['cleaned_review'] = df['review'].apply(preprocess_text)

sample = 'The movie was AMAZING!!! Best film of 2024. Visit www.movies.com'
print('Original:', sample)
print('Cleaned :', clean_text(sample))
print('Tokens  :', tokenize_text(clean_text(sample)))

for i in range(len(df)):
    print('\nOriginal:', df.loc[i, 'review'])
    print('Cleaned :', df.loc[i, 'cleaned_review'])

vectorizer = TfidfVectorizer(max_features=5000, ngram_range=(1, 2), min_df=1, max_df=0.95)
X = vectorizer.fit_transform(df['cleaned_review'])
print('\nShape of TF-IDF matrix:', X.shape)
print('Vocabulary size:', len(vectorizer.vocabulary_))
print('Sample feature names:', vectorizer.get_feature_names_out()[:10])

def get_top_words(df_subset, n=15):
    all_words = ' '.join(df_subset['cleaned_review']).split()
    return Counter(all_words).most_common(n)

pos_words = get_top_words(df[df['sentiment'] == 'positive'])
neg_words = get_top_words(df[df['sentiment'] == 'negative'])

fig, axes = plt.subplots(1, 2, figsize=(14, 6))
for ax, words, title in [(axes[0], pos_words, 'Top Words in POSITIVE Reviews'),
                         (axes[1], neg_words, 'Top Words in NEGATIVE Reviews')]:
    ws = [w[0] for w in words]
    cs = [w[1] for w in words]
    ax.barh(ws[::-1], cs[::-1])
    ax.set_title(title, fontsize=12, fontweight='bold')
    ax.set_xlabel('Frequency')
plt.tight_layout()
plt.savefig('top_words_by_sentiment.png', dpi=150)
plt.show()
```

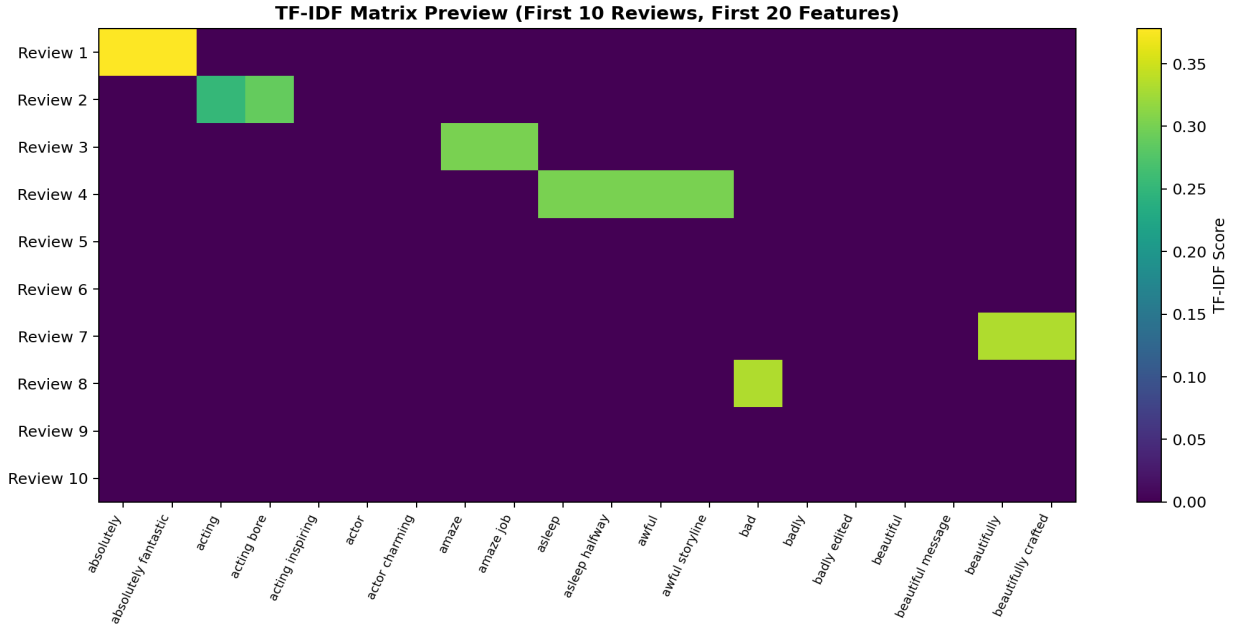
Output Screenshots

```
TASK 1 OUTPUT - TEXT PREPROCESSING AND TF-IDF
============================================================
Dataset shape: (16, 3)

Original sample: The movie was AMAZING!!! Best film of 2024. Visit www.movies.com
Cleaned sample : the movie was amazing best film of visit
Tokens         : ['the', 'movie', 'was', 'amazing', 'best', 'film', 'of', 'visit']
Without stopwords: ['amazing', 'best', 'visit']

Stemming and Lemmatization Example:
running      stem=runn       lemma=run
happily      stem=happi      lemma=happily
better       stem=better     lemma=good
studies      stem=study      lemma=study
caring       stem=car        lemma=care
wolves       stem=wolve      lemma=wolf

Original vs Cleaned Reviews:
1. Original: This movie was absolutely fantastic! Best film of the year.
   Cleaned : absolutely fantastic best year
2. Original: Terrible acting and boring plot. Complete waste of time!
   Cleaned : terrible acting bore plot complete waste time
3. Original: Loved every minute of it. The director did an amazing job.
   Cleaned : love every minute director amaze job
4. Original: Awful storyline. I fell asleep halfway through.
   Cleaned : awful storyline fell asleep halfway through
5. Original: Brilliant performances! The cinematography was breathtaking.
   Cleaned : brilliant performance cinematography breathtaking
6. Original: Worst movie I have ever seen. Do not waste your money.
   Cleaned : worst ever seen waste money

Shape of TF-IDF matrix: (16, 145)
Vocabulary size: 145
Sample feature names: absolutely, absolutely fantastic, acting, acting bore, acting inspiring, actor, actor charming, amaze, amaze job, asleep, asleep halfway

Top positive words: [('absolutely', 1), ('fantastic', 1), ('best', 1), ('year', 1), ('love', 1), ('every', 1), ('minute', 1), ('director', 1), ('amaze', 1), (
Top negative words: [('bore', 2), ('plot', 2), ('waste', 2), ('terrible', 1), ('acting', 1), ('complete', 1), ('time', 1), ('awful', 1), ('storyline', 1), ('f
```

Top Words by Sentiment



TF-IDF Matrix Preview



Written Report (250-350 Words)

Text preprocessing is the first and most important step in any Natural Language Processing project. Raw movie reviews contain punctuation, capital letters, numbers, website links, and many common words that do not help the model understand sentiment. In this task, the review text was cleaned and converted into a useful numerical format for machine learning.

First, regular expressions were used to remove HTML tags, URLs, punctuation, numbers, and extra spaces. The complete text was converted into lowercase so that words like Amazing and amazing are treated as the same word. After cleaning, tokenisation was applied. Tokenisation splits a sentence into individual words called tokens. For example, the sentence “the movie was amazing best film” becomes separate words such as movie, amazing, best, and film.

Next, stopwords were removed. Stopwords are common words like the, was, is, and a. These words appear very frequently but usually do not carry strong meaning for sentiment analysis. Removing them reduces noise and makes the important words clearer. Lemmatization was also used to bring words closer to their base form. This helps the system treat similar words as one concept.

Finally, TF-IDF vectorisation was applied. Machine learning algorithms cannot directly understand text, so TF-IDF converts each cleaned review into numbers. It gives higher value to words that are important in one review but not common in all reviews. This creates a sparse matrix where rows represent reviews and columns represent important words or phrases. Word frequency analysis also showed the most common positive and negative words, helping us understand the dataset before model training.

This task successfully prepared clean text features that can be used in Task 2 for building a sentiment classification model.

Conclusion

Task 1 is completed successfully. The raw review text was cleaned, converted into useful tokens, transformed with TF-IDF, and analyzed using word frequency visualization. These outputs are ready for the next sentiment classification step.